



Guia Mestra de Python: De Zero a Professional

1. Fonaments de Sintaxi i Tipus de Dades

Python es basa en la **llegibilitat**. A diferència d'altres llenguatges, no usa claus `{ }` per separar blocs de codi, sinó l'**indentació** (espais al principi de la línia).



Tipus de dades bàsics

Tipus	Nom	Exemple	Nota
Enter	<code>int</code>	<code>edat = 25</code>	Números sense decimals.
Flotant	<code>float</code>	<code>preu = 9.99</code>	Números amb decimals.
Cadena	<code>str</code>	<code>"Hola"</code>	Text entre cometes simples o dobles.
Booleà	<code>bool</code>	<code>es_cert = True</code>	Només pot ser <code>True</code> o <code>False</code> .
Nul	<code>NoneType</code>	<code>valor = None</code>	Representa l'absència de valor.



Operadors que cal conèixer

- **Aritmètics:** `+`, `-`, `*`, `/` (divisió decimal), `//` (divisió entera), `%` (mòdul/residu), (potència).
- **Comparació:** `==` (igual), `!=` (diferent), `>`, `<`, `>=`, `<=`.
- **Lògics:** `and`, `or`, `not`.

2. Col·leccions de Dades (Estructures)

Saber triar la col·lecció adequada és el que diferencia un principiant d'un expert.

Llistes (list)

Són ordenades i **mutables** (es poden canviar).

Python

```
fruites = ["poma", "plàtan"]
fruites.append("maduixa") # Afegeix al final
fruites[0] = "pera"      # Canvia el primer element
print(len(fruitess))     # Longitud: 3
```

Tuples (tuple)

Són ordenades però **immutablees**. S'usen per a dades que no han de canviar (com coordenades GPS).

Python

```
punt = (10, 20)
# punt[0] = 5 <-- Això donaria ERROR
```

Diccionaris (dict)

Guarden dades amb el format **clau: valor**. Són extremadament ràpids per buscar informació.

Python

```
usuari = {"nom": "Marc", "nivell": 10}
print(usuari["nom"]) # "Marc"
usuari["puntuacio"] = 50 # Afegeix nova clau
```

3. Control de Flux (Lògica)

Condicionals

Python

```
edat = 18
if edat >= 18:
    print("Ets major d'edat")
elif edat > 12:
    print("Ets adolescent")
else:
    print("Ets un infant")
```

Bucles

- **For:** Ideal per recórrer col·leccions o rangs.

Python

```
for i in range(5): # Genera 0, 1, 2, 3, 4
    print(f"Número: {i}")
```

```
noms = ["Anna", "Pau"]
for n in noms:
    print(n)
```

- **While:** S'executa mentre una condició sigui certa.

4. Funcions i Modularitat

Les funcions permeten reutilitzar codi i evitar repetir-se (principi DRY: *Don't Repeat Yourself*).

Python

```
def saludar(nom="usuari"): # "usuari" és un valor per defecte
    return f"Hola, {nom}!"
```

```
print(saludar("Laia")) # "Hola, Laia!"
print(saludar())      # "Hola, usuari!"
```

5. Ampliació: Programació Orientada a Objectes (OOP)

Per a projectes grans (com un videojoc a Unity o una App), usem **Classes**.

Python

```
class Jugador:
    def __init__(self, nom): # Constructor
        self.nom = nom
        self.salut = 100

    def rebre_dany(self, quantitat):
        self.salut -= quantitat
        print(f"{self.nom} té {self.salut} de vida.")
```

```
heroi = Jugador("Aragorn")
heroi.rebre_dany(20)
```

6. Trucs d'Expert (Nivell "Herói")

✨ List Comprehensions

Una manera elegant de crear llistes en una sola línia.

Python

```
# Volem els quadrats dels números del 1 al 5
quadrats = [x**2 for x in range(1, 6)]
# Resultat: [1, 4, 9, 16, 25]
```

Gestió d'Errors (Try/Except)

Evita que el teu programa "peti" si passa una cosa inesperada.

Python

```
try:
    num = int(input("Entra un número: "))
    print(10 / num)
except ZeroDivisionError:
    print("No pots dividir per zero!")
except ValueError:
    print("Has d'introduir un número vàlid.")
```

Llibreries imprescindibles

Python és famós pel seu ecosistema. Per "saber fer coses millors", aprèn a usar aquests mòduls:

- **os / pathlib**: Per manipular fitxers i carpetes.
- **requests**: Per connectar-te a internet i llegir dades de webs.
- **pandas**: Per analitzar grans quantitats de dades (Excel, CSV).
- **pygame**: Per crear jocs senzills en 2D.